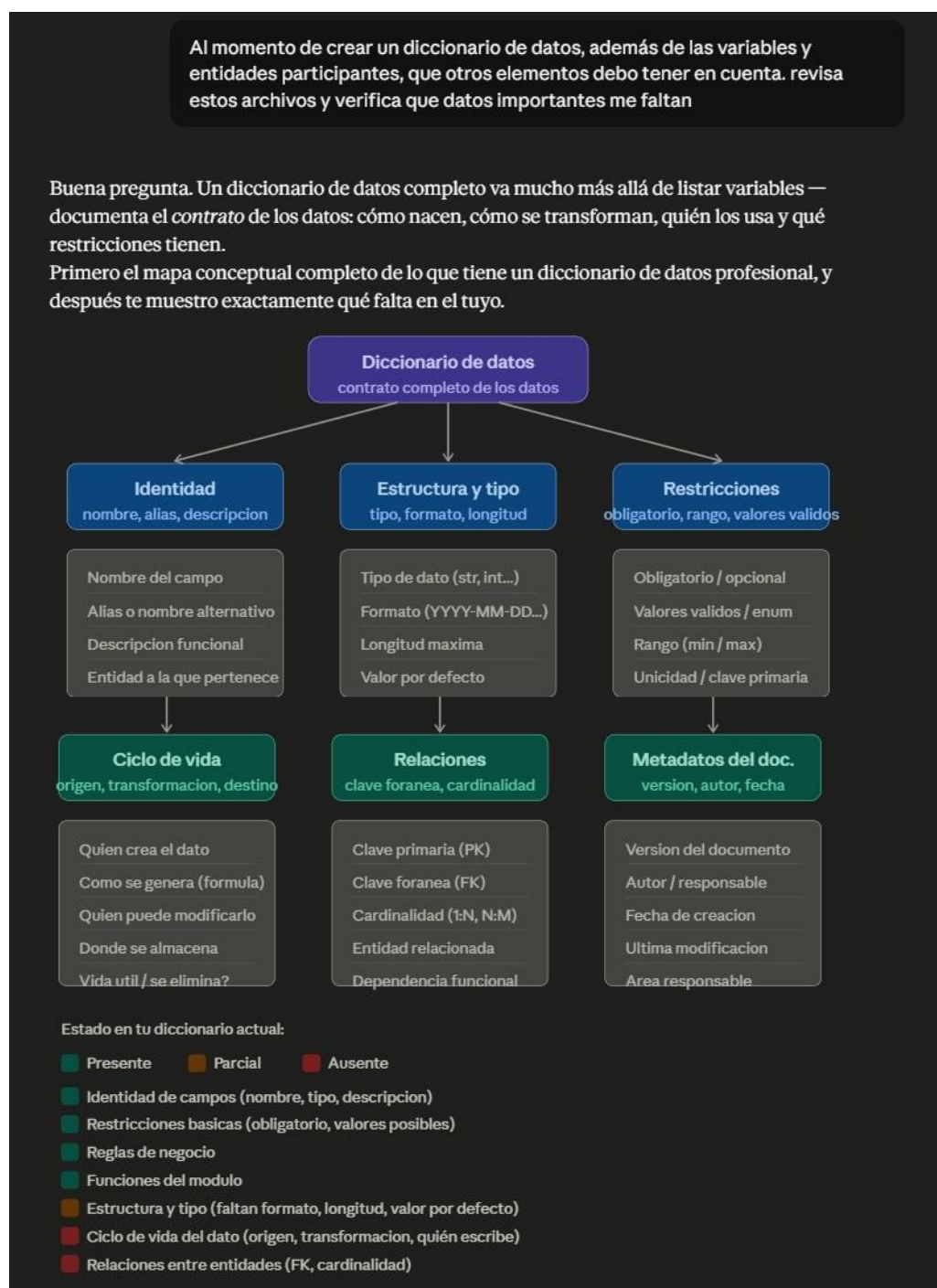


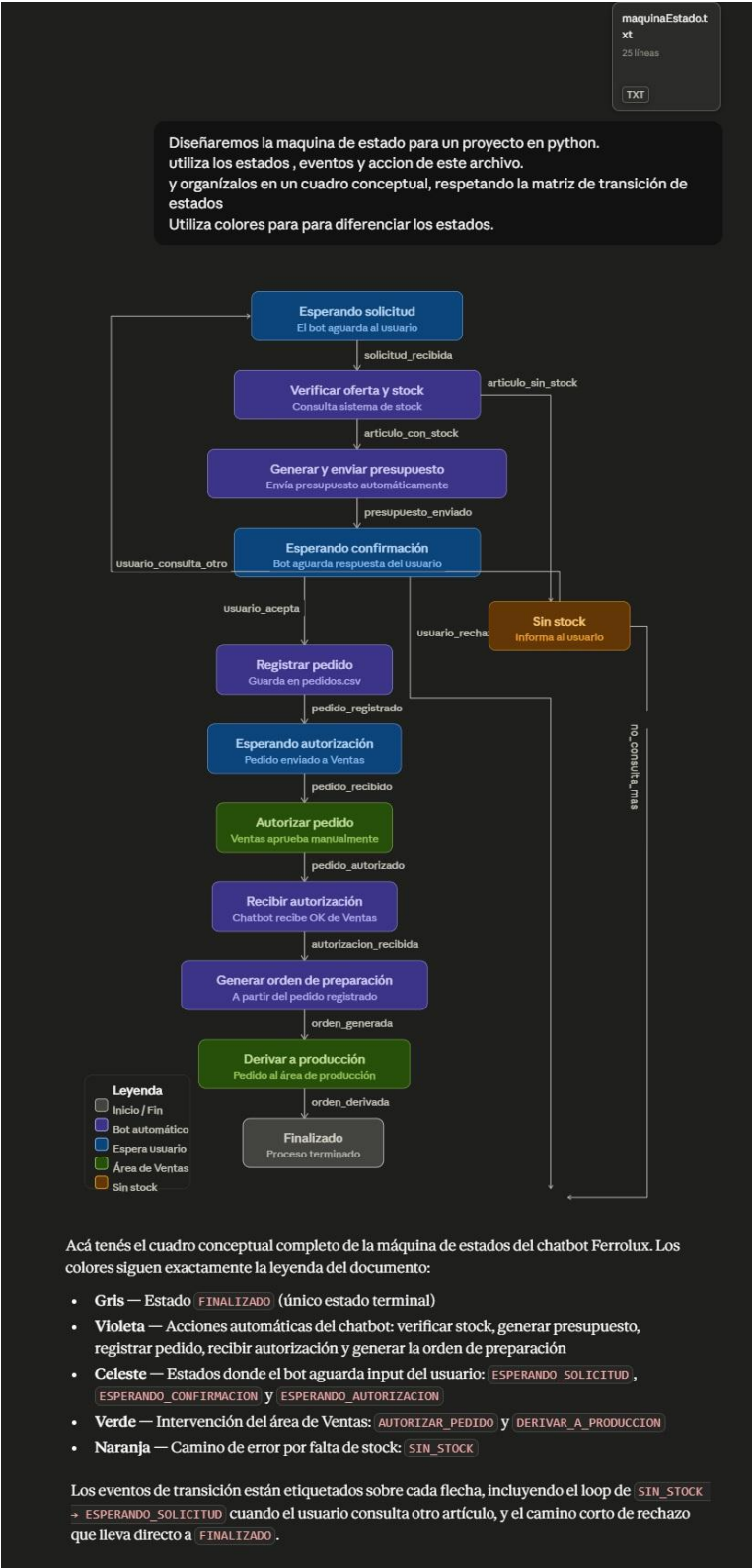
Anexo – Registro de Consultas Realizadas con Herramientas de Inteligencia Artificial

Este documento reúne las consultas efectuadas durante el desarrollo del trabajo práctico. Las evidencias se presentan en el mismo orden que fueron capturadas, indicando la herramienta utilizada y una breve descripción del objetivo de cada consulta.

Consulta 1 – Claude: Estructura del diccionario de datos



Consulta 2 – Claude: Estructura de Maquina de estado



Consulta 3 – Cursor: Estructura del código en Python

Maquina_de_estado.docx

to-be.bpmn

vamos a crear un código en python, utilizando la estructura match case, donde cada caso sean los estados declarados en el archivo maquina_de_estado.docx, debe recrear el flujo del archivo to-be.bpmn. como estructurarlo correctamente y que factores debo tener en cuenta? no escribas el código, solo un guía

Revisando ambos archivos para entender estados y flujo del BPMN y darte una guía concreta.

Read to-be.bpmn L1-413

Extrayendo el contenido del .docx para alinear la guía con los estados declarados.

Ran Extract docx XML via PowerShell powershell

Explored 3 files, 4 searches

Guía: estructurar la máquina de estados con match/case (FerroLux)

Basándome en tu to-be.bpmn y en la tabla de estados del documento (equivalente a Maquina_de_estado.docx), esta es la forma recomendada de pensarlo sin escribir código.

1. Aclarar qué es cada cosa (error más común)

No conviene que cada nodo del BPMN sea un case. El BPMN tiene más detalle que la máquina de estados.

Concepto en el docx	Rol en Python
Estado	Valor del case (un Enum)
Evento	Disparador de la transición (input del usuario, resultado de una verificación, etc.)
Acción	Función aparte que se llama desde el case
Gateway del BPMN (¿Hay stock?, ¿Aceptar?, etc.)	No es un estado: es una decisión que produce un evento

Correspondencia BPMN → estados del docx

```
graph TD
    ESPERANDO_SOLICITUD --> VERIFICAR_OFERTA_Y_STOCK
    VERIFICAR_OFERTA_Y_STOCK --> articulo_con_stock
    VERIFICAR_OFERTA_Y_STOCK --> articulo_sin_stock
    articulo_con_stock --> GENERAR_Y_ENVIAR_PRESUPUESTO
    GENERAR_Y_ENVIAR_PRESUPUESTO --> ESPERANDO_CONFIRMACION
    ESPERANDO_CONFIRMACION --> usuario_acepta
    usuario_acepta --> REGISTRAR_PEDIDO
    REGISTRAR_PEDIDO --> ESPERANDO_AUTORIZACION
    ESPERANDO_AUTORIZACION --> AUTORIZAR_PEDIDO
    AUTORIZAR_PEDIDO --> RECIBIR_AUTORIZACION
    RECIBIR_AUTORIZACION --> GENERAR_ORDEN_DE_PREPARACION
    GENERAR_ORDEN_DE_PREPARACION --> DERIVAR_A_PRODUCCION
    DERIVAR_A_PRODUCCION --> FINALIZADO
    articulo_sin_stock --> usuario_consulta_otro
    usuario_consulta_otro --> ESPERANDO_SOLICITUD
    articulo_sin_stock --> usuario_rechaza
    usuario_rechaza --> SIN_STOCK
    usuario_rechaza --> FINALIZADO
    SIN_STOCK --> usuario_no_consulta_mas
    usuario_no_consulta_mas --> FINALIZADO
```

2. Arquitectura recomendada (capas)

Separá responsabilidades en módulos pequeños:

```
main.py          → arranca el proceso, bucle principal
estados.py       → Enum de estados (los del docx, tal cual)
eventos.py       → Enum de eventos (columna "Evento" del docx)
contexto.py      → datos compartidos del pedido en curso
acciones.py      → una función por cada "Acción" del docx
transiciones.py  → tabla estado + evento → estado siguiente
maquina.py       → match/case que orquesta
```

Principio clave

El `match/case` no debe contener toda la lógica. Solo debe:

1. Ejecutar la acción del estado actual.
2. Obtener el evento resultante.
3. Calcular el siguiente estado.
4. Actualizar el contexto.

3. Cómo usar `match/case` correctamente

Patrón principal: un `case` por estado

```
while estado != FINALIZADO:
    match estado:
        case ESPERANDO_SOLICITUD: ...
        case VERIFICAR_OFERTA_Y_STOCK: ...
        ...
        case FINALIZADO: break
```

Cada `case` representa un estado declarado en el docx, no una tarea suelta del BPMN.

Qué va dentro de cada `case`

Dentro del <code>case</code>	Fuera del <code>case</code>
Llamar a la acción correspondiente	Validación de stock
Decidir qué evento ocurrió	Lectura/escritura de CSV
Devolver el siguiente estado	Cálculo de presupuesto y descuento
	Simulación del área de Ventas

Patrón secundario (opcional): `match` anidado para gateways

Para estados con bifurcación (`VERIFICAR_OFERTA_Y_STOCK`, `SIN_STOCK`, `ESPERANDO_CONFIRMACION`), podés:

- Opción A (recomendada): el `case` llama a una acción que retorna un evento, y una tabla de transiciones resuelve el siguiente estado.
- Opción B: `match` anidado `estado + evento` dentro del mismo bloque.

La opción A mantiene el `match` más limpio y alineado con la tabla del docx.

4. Los 12 estados y qué debe hacer cada uno

Estado	Tipo (leyenda docx)	Actor principal	Qué dispara la transición
ESPERANDO_SOLICITUD	Inicio	Usuario / Chatbot	Usuario ingresa producto y cantidad
VERIFICAR_OFERTA_Y_STOCK	Automático	Chatbot	Resultado de consulta a <code>productos.csv</code>
SIN_STOCK	Error / recuperación	Chatbot + Usuario	¿Quiere consultar otro artículo?
GENERAR_Y_ENVIAR_PRESUPUESTO	Automático	Chatbot	Presupuesto generado
ESPERANDO_CONFIRMACION	Espera usuario	Usuario	Acepta o rechaza
REGISTRAR_PEDIDO	Automático	Chatbot	Pedido guardado en CSV como <code>pendiente</code>
ESPERANDO_AUTORIZACION	Ventas	Ventas	Pedido recibido para revisión
AUTORIZAR_PEDIDO	Ventas	Ventas	Autorización manual
RECIBIR_AUTORIZACION	Automático	Chatbot	Autorización confirmada
GENERAR_ORDEN_DE_PREPARACION	Automático	Chatbot	Orden generada
DERIVAR_A_PRODUCCION	Automático	Chatbot	Pedido derivado
FINALIZADO	Fin	—	Sin transiciones

Consulta 4 – ChatGpt: Comparación Maquina de estado / Código Python

 **Maquina_de_estado.docx**
Documento

necesito comparar mi código con mi maquina de estado. debe reflejarse perfectamente:

Comparando tu código con la máquina de estados, diría que coincide en un **95%**. Hay algunos detalles que podrías ajustar si el profesor espera una correspondencia estricta.

Coincidencias correctas

Todos estos estados existen en ambos:

- ESPERANDO_SOLICITUD
- VERIFICAR_OFERTA_Y_STOCK
- SIN_STOCK
- GENERAR_Y_ENVIAR_PRESUPUESTO
- ESPERANDO_CONFIRMACION
- REGISTRAR_PEDIDO
- ESPERANDO_AUTORIZACION
- AUTORIZAR_PEDIDO
- GENERAR_ORDEN_DE_PREPARACION
- DERIVAR_A_PRODUCCION
- FINALIZADO

Y las transiciones principales coinciden correctamente.  Maquina_de_estado

Diferencia 1: Falta el estado RECIBIR_AUTORIZACION

En la máquina:

AUTORIZAR_PEDIDO
↓
RECIBIR_AUTORIZACION
↓
GENERAR_ORDEN_DE_PREPARACION



 Maquina_de_estado

En tu código:

 **Python**   Ejecutar

```
case "AUTORIZAR_PEDIDO":  
  
    if opcion == "1":  
        estado = "GENERAR_ORDEN_DE_PREPARACION"
```



Solución

Agregar:

 **Python**   Ejecutar

```
case "AUTORIZAR_PEDIDO":  
  
    if opcion == "1":  
        estado = "RECIBIR_AUTORIZACION"  
  
    else:  
        print("Pedido rechazado por Ventas.")  
        estado = "FINALIZADO"  
  
case "RECIBIR_AUTORIZACION":  
  
    print("Autorización recibida desde Ventas.")  
  
    estado = "GENERAR_ORDEN_DE_PREPARACION"
```

Diferencia 2: Acción de REGISTRAR_PEDIDO

La máquina dice:

Registrar pedido en base de datos (pedidos.csv) con estado "pendiente".  Maquina_de_estado

Tu código registra:

```
</> Python  Ejecutar  
  
{  
    "producto": ...,  
    "cantidad": ...,  
    "presupuesto": ...  
}
```

No guarda el campo:

Si querés ser totalmente fiel al modelo:

```
</> Python  Ejecutar  
  
columnas = [  
    "producto",  
    "cantidad",  
    "presupuesto",  
    "estado"  
]  
  
escriptor.writerow({  
    "producto": fila['producto'],  
    "cantidad": cantidad,  
    "presupuesto": presupuesto,  
    "estado": "pendiente"  
})
```

Diferencia 3: El nombre del archivo

La máquina menciona:

pedidos.csv 

 Maquina_de_estado

Tu código actual usa:

```
</> Python  Ejecutar  
  
pedidos_registrados.csv
```

No es un error funcional, pero si el BPMN y la máquina dicen `pedidos.csv`, conviene usar el mismo nombre para mantener trazabilidad.

Consulta 5 – Claude: Diseño Manual de usuario

como diseñar correctamente un manual de usuario. basado en los datos de estos archivos?
muéstrame como armar una estructura y como ordenar su contenido

10:50 a. m.  

Buena pregunta — un manual de usuario tiene una estructura bastante estándar, pero el orden y el nivel de detalle dependen de *quién lo va a leer y para qué lo va a usar*. Basándome en FerroLux y los archivos que ya tenemos, te muestro cómo pensarlo:

Podés hacer clic en cada sección para ver más detalle. Acá te explico la lógica detrás del orden:

El principio central: de lo general a lo específico, del usuario al sistema

Un manual bien estructurado responde estas preguntas en orden:

1. ¿Qué es esto y para qué sirve? → Introducción
2. ¿Puedo usarlo en mi computadora? → Requisitos
3. ¿Cómo lo uso yo (cliente)? → Pasos del cliente
4. ¿Cómo lo uso yo (ventas)? → Pasos de ventas
5. ¿Qué productos maneja? → Catálogo
6. ¿Qué pasa si algo sale mal? → Errores
7. ¿Cómo se ve en la práctica? → Ejemplo completo
8. ¿Qué significa tal término / qué hago si tengo más dudas? → Glosario, FAQ, soporte

Por qué ese orden para FerroLux en particular:

El chatbot tiene dos actores distintos (cliente y ventas) que usan el sistema en momentos diferentes. Por eso el manual separa los flujos en secciones independientes, en lugar de mezclarlos. El cliente no necesita leer lo de ventas, y viceversa.

El catálogo va después de los flujos porque es información de consulta, no de aprendizaje. Y los errores van al final de la parte operativa porque el usuario los busca solo cuando algo falla, no al aprender por primera vez.

¿Querés que genere el documento Word completo con esta estructura aplicada a FerroLux, o preferís ajustar alguna sección antes?

Consulta 6 – Claude: Diseño y estructura de informe final



Como estructurar correctamente el PDF de entrega con respecto a este archivo, el cual tendrá los contenidos de: descripción de una empresa, descripción de un proceso ineficiente, el objetivo de la mejora de nuestro Chatbot, descripción de los diagramas AS-IS y TO-BE, explicación sobre las guías usadas y links al repositorio de GitHub.

Leyendo la guía con cuidado, el PDF debe cubrir exactamente estos puntos del "Contenido a evaluar":

1. Definición del proceso + diagramas AS-IS y TO-BE
2. Configuración del entorno (GitHub) + esqueleto del bot
3. Integración con base de datos + lógica de decisiones
4. Capturas de consultas a IA

Con lo que vos querés incluir, te propongo esta estructura para las 4-8 páginas:

Página 1 — Carátula

Nombre del trabajo, organización (FerroLux), integrantes, cátedra, fecha.

Página 2 — La organización y el problema

- Descripción de FerroLux y el área de Ventas
- El proceso actual ineficiente (AS-IS en texto)
- Objetivo de la mejora con el chatbot

Página 3 — Diagramas BPMN

- Diagrama AS-IS (imagen con aclaración centrada debajo)
- Diagrama TO-BE (imagen con link al `.bpmn` centrado debajo)

Página 4 — Arquitectura técnica

- Plataforma y lenguaje elegidos + justificación (consola/simulador, Python 3.10+)
- Base de datos: explicación de `productos.csv` y `pedidos_registrados.csv`
- Lógica de decisiones: los gateways del BPMN y cómo se implementaron con `match/case`

Página 5 — Gestión de estados y robustez

- Resumen de la máquina de estados (tabla)
- Camino infeliz: tabla de errores y respuestas del sistema

Página 6 — Herramientas de IA y repositorio

- Capturas de pantalla de consultas realizadas a IA con sus respuestas
- Link al repositorio de GitHub con descripción breve de su contenido

Así quedás dentro de las 6-7 páginas, respetás todas las normas de formato y cubrís cada punto del contenido a evaluar. ¿Querés que arranque a armar el PDF con esta estructura?

